

Enforcing cryptographic distributed-VCS access control with no trust on servers

Xin Xu ^{a,*}, Zhen Yang ^b, Quanwei Cai ^c, Jingqiang Lin ^d, Liangqin Ren ^e, Bo Chen ^f,
Yongfeng Huang ^a

^a Department of Electronic Engineering, Tsinghua University, 100084, Beijing, China

^b School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing, 100876, China

^c Beijing Zitiao Network Technology Co., Ltd, Beijing, 100098, China

^d University of Science and Technology of China, Hefei, 230022, China

^e Department of Electrical Engineering and Computer Science, University of Kansas, Lawrence, 66045, USA

^f Department of Computer Science, Michigan Technological University, Houghton, 49931, USA

ARTICLE INFO

Keywords:

Access control
Distributed system
Version control
Git
Attributed-based cryptography
Smart contract

ABSTRACT

Version control systems (VCS), including central VCS (CVCS) and distributed VCS (DVCS), are widely adopted to manage changes to software code and various types of documents. Unlike CVCS, where entities obtain data from a central server, each entity in DVCS stores the entire repository and shares it independently. In VCS, existing access control schemes require the participation of a central server and cannot be deployed in a completely distributed scenario. Additionally, these schemes often fail to enforce fine-grained access control for write permissions, which is crucial for collaborative work in a distributed environment. In this paper, we propose a distributed version control system access control scheme (named DVAC), which enforces cryptographic access control on distributed user nodes based on attribute-based encryption (ABE) and attribute-based signature (ABS). DVAC is designed to enforce a cryptographic access control protocol for DVCS, which enables file granularity read and write separation access control without the support of a central server. To ensure the integrity of the core version control functions in DVCS while protecting data security, DVAC incorporates a version control adaptation protocol. Additionally, DVAC leverages Ethereum smart contracts to maintain access control policies, ensuring distributed storage and trusted management of access policies. The architecture of DVAC is designed to seamlessly integrate with existing mature DVCS, such as Git, with minimal modifications. We have implemented a prototype of DVAC and integrated it with Git. A comprehensive performance evaluation was conducted to assess the overhead introduced by DVAC, and it was demonstrated that the overhead is modest.

1. Introduction

Version control systems (VCS) are crucial for managing data changes and enabling project collaboration. They can be standalone or built-in. VCS can be categorized as central (CVCS) or distributed (DVCS). CVCS, like CVS [1] and SVN [2], rely on a central server for reference versions, with clients synchronizing their modifications. DVCS, such as Git [3] and Veracity [4], use a peer-to-peer approach, where each peer maintains its own repository with a complete change history and synchronizes with other peers. DVCS are more efficient and flexible than CVCS, as they eliminate single points of failure. In DVCS, each entity works on its local repository and performs common operations without communication. Entities can create, maintain, or delete local branches independently. They can also choose which modifications to

share and store copies pulled from others, providing data backup and avoiding single points of failure.

DVCS introduce novel challenges to data access control. In this decentralized system, peers have the ability to exchange modifications directly, circumventing the need for a central server to enforce access control policies. Moreover, collaboration in DVCS necessitates the segregation of read and write permissions, thereby amplifying the complexity of access control. The intricacies of access control in DVCS can be succinctly encapsulated as follows: (1) **Inability to depend on a central server**; (2) **Uniform implementation across all peers**; (3) **Separate control of read and write permissions**.

Existing access control schemes for cloud storage and CVCS [5–9] are not applicable for DVCS because they require the cooperation of a

* Corresponding author at: Department of Electronic Engineering, Tsinghua University, 100084, Beijing, China.

E-mail address: xuxin527@mail.tsinghua.edu.cn (X. Xu).

central server to enforce access control. For example, schemes based on proxy re-encryption [6,7] and attribute-based cryptosystems [5,8,9] all rely on a server for various tasks such as re-encryption, verification, and signature checking. Distributed access control schemes [10–14] also do not meet the requirements of DVCS, as they lack support for write permission control, which is essential for collaboration. These schemes primarily focus on preventing unauthorized read access to sensitive data. However, they are not suitable for DVCS. The distributed read and write framework Disac [15] lacks efficient metadata management and experimental verification. Current access control schemes for Git [16–19] also face challenges in extending to distributed scenarios, lack write access control, rely on a central server, or involve excessive key maintenance overhead. Therefore, none of the existing access control schemes can effectively address the access control difficulties in DVCS.

In this paper, a novel distributed access control scheme named DVAC is introduced, specifically tailored for DVCS. DVAC utilizes attribute-based encryption (ABE), attribute-based signature (ABS), and smart contracts to achieve its objectives. The contributions of this work are summarized as follows:

- DVAC provides enforced cryptographic access control for a fully distributed VCS. DVAC can consistently control access to files on distributed user nodes without requiring a central server's involvement. DVAC provides granular access control for each file, with permissions specified by the data owner to define the data scope and its corresponding access control policies. The access control policy is managed in a distributed and trusted manner by an Ethereum-based smart contract, ensuring consistent multi-node control and distributed access control implementation.
- DVAC provides a read and write separation access control protocol and a version control adaptation protocol for DVCS, ensuring data security without disrupting the normal functions of DVCS. We implement read–write separation access control based on attribute-based encryption (ABE) and attribute-based signature (ABS). Users whose attributes do not satisfy the read policy cannot decrypt the ciphertext associated with the data, and modifications made by users whose attributes do not satisfy the write policy are rejected and not accepted by other users. DVAC offers an adaptation protocol for the core version control function of DVCS, ensuring cryptographic protection for data in the repository and on the network, without affecting the version control operation in the work-space. As a result, DVAC prevents unauthorized manipulation of data stored by individual peers without undermining the efficient and flexible version control and collaboration provided by DVCS.
- We implemented the DVAC prototype integrated with Git, offering trusted metadata management combined with the Ethereum blockchain and lightweight key management, regardless of the number of users. DVAC's architecture is highly compatible with existing DVCS and requires minimal configuration. Each access control policy is uniquely associated with the data (i.e., files), and a smart contract is utilized to ensure that the access control policy remains under the control of the data owner while being accessible to all entities. We have successfully implemented a prototype of DVAC and integrated it with Git, demonstrating its practical application. The performance evaluation of DVAC indicates that the introduced overhead is within acceptable limits for practical usage scenarios.

The rest of the paper is organized as follows. Section 2 introduces the background and related works. We present the system overview and threat model in Section 3, and detail the specific protocols and algorithms of DVAC in Section 4. Then, we describe how to integrate DVAC with Git in Section 5. The security is analyzed in Section 6, and the implementation details and evaluation are provided in Section 7. Finally, Section 9 concludes this paper.

2. Background and related works

DVAC employs ABE and ABS to enforce access control for read and write operations in DVCS. Furthermore, smart contracts are utilized in DVAC to guarantee the availability and integrity of the access control policy. This section provides crucial background information on DVCS, ABE, ABS, and smart contracts, and concludes with a summary of pertinent research in the field.

2.1. Distributed version control systems

DVCS improve project collaboration by providing participants with a complete repository and the ability to make local modifications. This flexibility empowers users to revise data locally, choose modifications to publish, and perform changes based on shared versions. In the absence of a central server, the project is redundantly stored across multiple participants, thereby eliminating single points of failure.

Git, a widely used DVCS, stores a snapshot of each version (called commit) instead of just the differences between versions. Git comprises four primary components: workspace, stage, local repository, and remote repositories. Users manipulate files in the workspace, stage modifications using `git add`, create new versions with `git commit`, and share modifications by pushing to remote repositories with `git push`. GitHub serves as a popular remote repository. Users employ commands such as `git clone` or `git fetch` to obtain repositories, `git branch` to create or delete branches, `git checkout` to switch branches or restore files, and `git merge` to combine versions. These operations are executed using the Git client.

2.2. ABE and ABS

ABE ensures that only entities with attributes satisfying a specified policy can successfully decrypt data [20]. Various ABE algorithms have been proposed, which can be classified into two categories: Key-Policy ABE (KP-ABE) and Ciphertext-Policy ABE (CP-ABE).

In KP-ABE [21], access control policies are related to users' secret keys. In CP-ABE [22], the user's secret key is associated with their attributes, and the data owner specifies the access control policy in the ciphertext. The ciphertext can be decrypted by users whose attributes satisfy the specified policy. CP-ABE provides the data owner complete control over the data and is used in DVAC to enforce read access control. In recent years, a variety of CP-ABE algorithms [23,24] adapted to practical scenarios have been proposed.

CP-ABE¹ consists of four functions: *ABE.Setup*, *ABE.KeyGen*, *ABE.Encrypt* and *ABE.Decrypt*.

- *ABE.Setup*(1^λ): Takes a security parameter 1^λ as input and outputs a public parameter PP_r and a master key MK_r .
- *ABE.KeyGen*(MK_r, S_i): Takes the master key MK_r and an attribute set S_i of user i as input, and outputs a secret key SK_r .
- *ABE.Encrypt*(PP_r, AP_r, m): Takes the public parameter PP_r , an access policy AP_r , and a message m to be encrypted as input, and outputs the ciphertext C_m .
- *ABE.Decrypt*(PP_r, SK_r, C_m): Takes the public parameter PP_r , a secret key SK_r , and a ciphertext C_m as input, and outputs the decrypted result m' .

ABS [25] ensures that only entities with attributes satisfying the specified access control policy can generate valid signatures. ABS consists of the following four algorithms.

- *ABS.Setup*(1^λ): Takes a security parameter 1^λ and returns a public parameter PP_w and a master key MK_w .

¹ Hereafter, ABE refers to CP-ABE for simplicity.

- $ABS.KeyGen(MK_w, S_i)$: Takes the master key MK_w and an attribute set S_i for user i and returns a secret key SK_w .
- $ABS.Sign(PP_w, AP_w, SK_w, m)$: Takes the public parameter PP_w , an access policy AP_w , secret key SK_w and a message m , and returns a signature σ .
- $ABS.Verify(PP_w, \sigma, m)$: Takes the public parameter PP_w , a signature σ and a message m . Returns 1 if the signature is valid, otherwise returns 0.

2.3. Smart contract

Smart contracts, introduced by Nick Szabo in 1994, are “computerized transaction protocols that execute the terms of a contract” [26]. They aim to fulfill common contractual conditions (e.g., payment terms, liens) without the need for intermediaries [27].

Smart contracts are facilitated by blockchain platforms like Ethereum [28] and Hyperledger Fabric [29]. As a widely deployed blockchain, Ethereum enables developers to create smart contracts using languages like Solidity, compile them, and deploy them. The smart contracts are stored and executed on the Ethereum virtual machine. To deploy a smart contract, the owner initiates a transaction, creating a contract account. Any Ethereum account interacts with the smart contract through the contract account. When an Ethereum account invokes a function to update data stored on the Ethereum network, it submits a corresponding transaction, which is recorded on the Ethereum network.

2.4. Related works

In this section, we divide the work into three categories: centralized access control schemes, distributed access control schemes, and access control schemes for Git.

Centralized access control schemes. In centralized access control schemes, a central server stores access control-related information (such as access control lists) and participates in the execution of controls. However, these schemes cannot scale to distributed scenarios because of the significant overhead imposed on access control enforcement entities. For example, in the scheme [6,7] based on proxy re-encryption, the entity needs to perform independent re-encryption for each version and each delegated user. Similarly, in schemes based on selective encryption [30,31], the entity needs to dynamically maintain the key derivation graph, which is not suitable for a larger number of cooperators. In contrast, DVAC only requires each entity to perform one ABE encryption and ABS signature for each version, regardless of the number of cooperators.

Distributed access control schemes. Distributed access control schemes like ABE, ABS, and smart contracts have been used for fine-grained access control on distributed data. However, these schemes are not suitable for DVCS, as they either require a central server to assist and cannot meet the fully distributed requirements [5,8,9,32], or they cannot provide write access control and do not support user collaboration [10–13].

Both schemes [8,9] propose access controls for electronic health records (EHR). RVWABE-CA [8] introduces a weighted attribute-based encryption scheme with revocation and verifiability, while CP-ABSC-MIoT [9] offers a privacy-preserving access control scheme based on attribute signcryption. However, both focus solely on read permissions for EHR and omit write permission control. Additionally, both depend on a central Verification Server or Authorization Engine, limiting flexibility in distributed environments. Similarly, the schemes [5,32] use ABE and ABS for data confidentiality and integrity, but also rely on a central server for revision checks.

BadS [13] uses smart contracts for access control in the Internet of Things (IoT), while Droplet [10] develops a decentralized authorization service for fine-grained access control. Exceline et al. [14] propose an EHR access control mechanism based on smart contracts, but the plaintext data remains unprotected during the smart contract-assisted

operations. Yang et al. [11] introduce an attribute-based access control scheme for IoT using blockchain technology. Li et al. [12] present an efficient ciphertext-policy weighted attribute-based encryption scheme for multi-user collaborative access in cloud storage. However, these schemes mainly focus on confidentiality, lack support for write access control, and are not applicable to DVCS where read and write permissions are separate. Xu et al. [15] propose a simple distributed read/write framework, but metadata is protected only by digital signatures, requiring users to maintain the public key certificates of data owners. Additionally, the application and evaluation in real DVCS scenarios are not addressed.

Access control schemes for Git. Several schemes have been proposed for securing Git. Gitolite [16] provides access control for Git in a central server model. Git-crypt [17] encrypts sensitive information in public repositories using Git filters [33]. Git-remote-gcrypt [18] encrypts entire repositories, distributing decryption keys through GPG. GitHub Enterprise [34] provide access control services based on the role-based access control (RBAC) model. Keybase-git [35] is built on the Keybase Encrypted File System (KBFS), and uses the ACL model for access control. In 2023, Xu et al. proposed Gringotts [19], an end-to-end encrypted VCS with file-level read access control using ABE and branch-level write access control using ECDSA. However, Gringotts relies on a remote server for maintaining an encrypted shadow repository and lacks flexibility in DVCS scenarios. DVAC integrates with Git through Git filters, providing fine-grained protection for each file in the repository. Key management in DVAC is greatly simplified because users only need to maintain their own read and write attribute keys, and data owners only need to set access policies based on attributes, without managing other users' related data.

3. System overview

For DVCS with distributed data storage, determining how to enforce and maintain consistent access control for distributed nodes is one of the most important challenges. The access control scheme should not require the participation of the central server, forcing the protection of file data on each distributed node, and the current access control scheme requires the participation of the server, which is easily bypassed by the node in DVCS.

We summarized the following three difficulties:

1. Enforce cryptographic access control across all DVCS nodes and separately control read and write permissions;
2. Access control must be fully compatible with DVCS features such as version control;
3. Access control-related data (such as access policies) requires distributed and secure storage and trusted management to provide consistent access control across multiple nodes.

In distributed version control scenarios, DVAC is designed to eliminate the user's reliance on a central server while ensuring accurate fine-grained read/write access control. Based on this premise, our objective is to minimize alterations and impact to existing DVCS and achieve seamless integration and user-friendliness of DVAC without requiring data migration or changes to normal operations. With these design goals in mind, we present a system overview of DVAC in this section.

As shown in Fig. 1, DVAC comprises three modules: enforcing cryptographic access control (EAC), version control adaptation (VCA) and trusted metadata management (TMM). DVAC involves the smart contract deployed on the blockchain, distributed user repositories and attribute authorities. Attribute authorities are responsible for user registration, key distribution and maintenance (such as PP), and are associated with the initial deployment of DVAC, which is not detailed here. The smart contract is deployed on the blockchain and is responsible for managing and maintaining the metadata version to ensure

Table 1
Symbol table.

Symbol	Description
SK_i^r	The read key of user i , used for ABE.Dec.
SK_i^w	The write key of user i , used for ABS.Sign.
ID_f	The unique identifier of the file.
ID_o	The unique identifier of a data owner.
AP_f^r	Read access policy for file f .
AP_f^w	Write access policy for file f .
$Meta_f$	Metadata for file f .
$Cipher_f$	Ciphertext of the file f content.
Sig_f	ABS signature of the file f .
$List_u$	The collection of mappings between ID^o s and blockchain accounts.
$List_f$	The collection of all ID^f s created.
$List_{AP}$	The collection of all metadata (including historical versions) indexed by ID^f s.

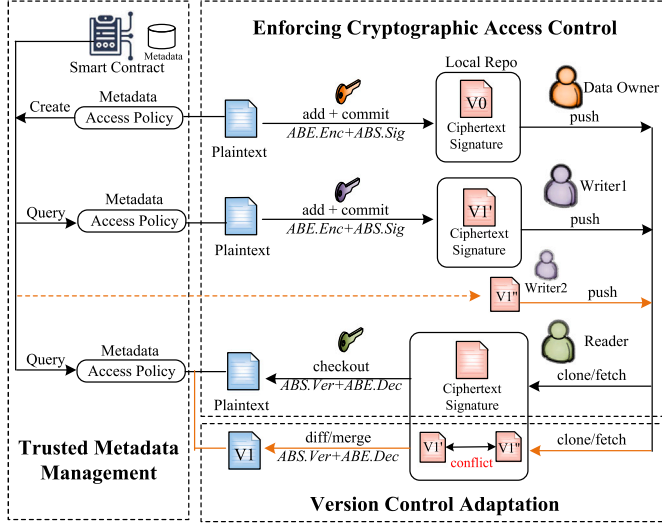


Fig. 1. The architecture of DVAC.

its correctness and validity. Users have a complete repository on their local storage, which is a distributed node in the DVCS. Following local modifications, users can opt to push them to other repositories. Users possess varying permissions for different files, categorized as data owners and readers/writers. Data owners have full control over the files they create and can specify and modify access policies, while readers/writers need to choose to read or modify data according to their own permissions. (See Table 1)

The EAC module is responsible for providing enforcement and consistent access control for files stored in distributed user repositories within DVCS. It also safeguards the historical versions of files stored in the user's local repository, which contain file ciphertexts and signatures. Ensuring that the original DVCS version control function is not affected when the files in the repository are all ciphertext is a crucial problem to address. We designed the VCA module to ensure the accuracy of DVCS version control of encrypted files. When users execute version control commands, appropriate data conversion and adjustment are performed to ensure the accurate execution of commands. The TMM module is essential for ensuring proper access control. It manages versions of metadata (including access policies) and maintains metadata security.

The attribute authority is responsible for distributing attribute keys to users. As shown in Fig. 1, the data owner creates a plaintext file $V0$, encrypts and signs it according to the access policy, and stores it in the local repository. In this case of cryptographic protection, the file can be shared with other users, who can then perform access control operations based on access policies and their own attribute keys. The metadata containing the access policy is managed by the

smart contract, and users have the capability to request metadata from the smart contract at any time during access control operations. Suppose that two modifiers create the $V1$ version of a file at the same time, which is called $V1'$ and $V1''$. This will cause version conflicts and require version control operations such as merging and comparing. DVAC will retrieve files in plaintext via the version control adaptation module for further DVCS operations.

Threat Model. In DVAC, each AA is trusted and securely stores sensitive information, such as MK_r , MK_w , and user secret keys. AA authenticates users and verifies their attributes before generating their keys. Public repositories hosting the data are honest-but-curious, storing all versions correctly but potentially performing unauthorized read or write operations. Valid users verify received versions using ABS signatures, decrypt data with their secret keys (SK_i^r), and generate valid ABS signatures for modified data using their secret keys (SK_i^w). Malicious users may engage in unauthorized activities, such as attempting to infer unauthorized data, uploading invalid modifications, or launching DDoS attacks. Collusion among malicious users is also possible to gain additional privileges.

4. DVAC scheme design

This section provides a detailed introduction to the specific design and protocol process of DVAC, along with proposed mitigation ideas for the challenging problems raised in the previous section.

Before delving into the specific protocols of DVAC, we establish the symbolic representation of the basic cryptographic algorithms as follows:

- $C \leftarrow Enc(M, k)$: Use the key k to perform symmetric encryption (e.g., AES) on the plaintext M to obtain the ciphertext C .
- $M \leftarrow Dec(C, k)$: Use the key k to perform symmetric decryption on the ciphertext C to obtain the plaintext M .
- $s \leftarrow Sig(M, sk)$: Asymmetric cryptographic algorithm (such as RSA). Use the private key sk to sign the message M to obtain the signature s .
- $0/1 \leftarrow Verify(s, M, pk)$: Use the public key pk to verify the signature s , output 1 means the signature is valid, otherwise it is invalid.

4.1. Initialization phase

During the initialization phase of DVAC, the $ABE.Setup$ and $ABS.Setup$ procedures are initially performed by the attribute authority to generate the associated public parameters (PP_r, PP_w) and master keys (MK_r, MK_w), which is followed by the user registration process. Each user needs to register with a trusted authority to obtain an authenticated identity ID. The protocol of register is as follows:

In real-world application scenarios, there are typically one or more trusted institutions responsible for verifying the user's actual identity in order for the user to register an account. The trusted institution provides the user with a DVCS account containing a verifiable ID.

1. The trusted institution generates a pair of signature keys (sk_a, pk_a) ;
2. Authenticate the user ID and generate a pair of signature keys (sk_{ID}, pk_{ID}) for the user;
3. Distribute $(sk_{ID}, cert)$ for the registered user ID , where $cert = (Info_{ID}, s) = ((ID, pk_{ID}), s)$, $s = Sig(Info_{ID}, sk_a)$.

In DVAC, users collaborate securely in DVCS with the help of blockchain. As a result, users go through the registration process using the blockchain smart contract.

1. The user ID creates a blockchain account Acc ;
2. The user ID initiates a registration request $Req(reg, Acc, ID, cert, s')$ to the smart contract, where $s' = Sig(Acc|cert, sk_{ID})$.
3. The smart contract verifies s , $0/1 \leftarrow Verify(s, Info_{ID}, pk_a)$. If 0 is returned, the error code “cert error” is printed.
4. The smart contract verifies s' , $0/1 \leftarrow Verify(s', Acc|cert, pk_{ID})$. If 0 is returned, the error code “user error” is printed.
5. The smart contract adds $(Acc, ID) \rightarrow List_u$.

In DVCS, the data are stored in the form of files, and DVAC adds extra information for each file to enforce the access control. Firstly, we introduce file-related data structures designed in DVAC.

- **File ID:** The identity of each file (f) need to be unique and easily derivable for users. In DVAC, the file ID ID^f is defined as $ID^f = (Init_CID|R_Name|F_Path)$, where $Init_CID$ is the digest of the first commit information (e.g., creator, creating time, included files, etc.) of the repository, and F_Path is the relative file path. $Init_CID$ and R_Name can uniquely refer to a repository, and F_Path can uniquely specify a file in the repository.
- **Metadata:** Metadata refers to information about file and access policies. We define the metadata of a file as $Meta_f$ is $Meta_f = (ID^o, ID^f, AP_r^f, AP_w^f)$, where ID^o and ID^f are the unique identity of the data owner and the file, and AP_r^f and AP_w^f are the read and write access policies of the file, respectively. $Meta_f$ is indexed by ID^f , bound to file (f) one by one, and the rest part ID^o, ID^f, AP_r^f and AP_w^f can only be modified by the data owner ID^o .
- **File:** In DVAC, each file (f) is stored in the form of $[Meta_f, Cipher_f, Sig_f]$, where $Cipher_f$ contains the ciphertext of f , while Sig_f is the ABS signature of $Cipher_f$.

Here, we establish a unique and easily generated ID^f for each file to ensure that the user can accurately index the metadata using the ID^f , thereby avoiding confusion resulting from ID duplication and data owner preemption.

4.2. Enforcing cryptographic access control (EAC)

In DVAC, users only need to possess their own pair of read and write attribute keys for file-level access control in various repositories. This reduces the burden of key management for users. In DVCS environments, files resemble blockchain ledgers and can be stored in a distributed manner in local repositories of each participating user. Users who are unable to decrypt files using the read attribute key do not have read access, and similarly, users who are unable to generate valid signatures using the write attribute key do not have write access. We describe the EAC module in detail below, including EAC protocols and adjustments to basic DVCS file operations.

As shown in Fig. 2, when creating a file, the data owner needs to set the read and write access policies (AP_r , AP_w) for the file. DVAC will generate the file ID ID^f and synthesize

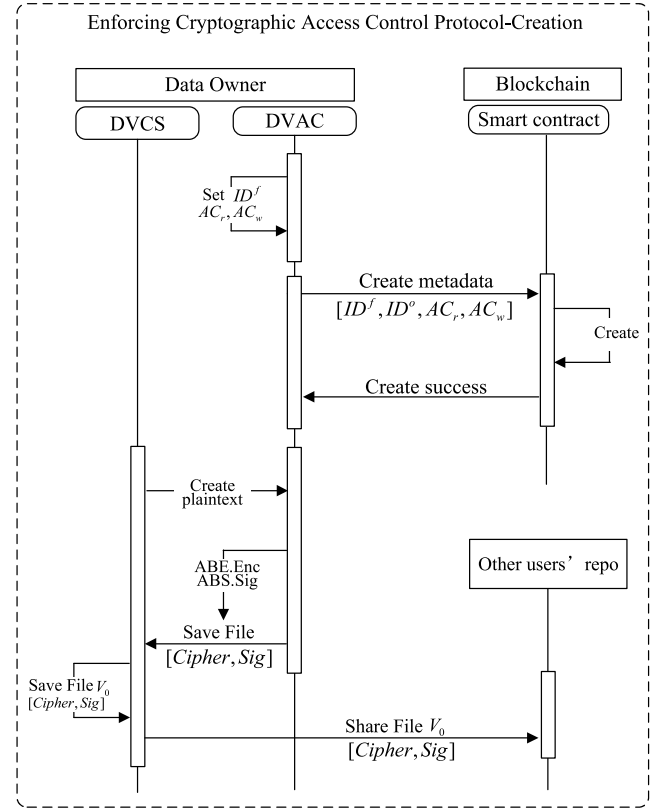


Fig. 2. Enforcing cryptographic access control protocol with file creation.

the metadata $(ID^o, ID^f, AP_r^f, AP_w^f)$ to initiate a metadata creation request to the smart contract. The metadata creation process can be synchronized with the process of the data owner writing the contents of the file. Upon completion of both processes, DVAC performs $ABE.Enc$ and $ABS.Sign$ according to the read and write access policy and the attribute key (SK_r, SK_w) , and stores the generated $[Cipher, Sig]$ in the local repository of the data owner. The data owner has the option to share the file $[Cipher, Sig]$ with other users.

Then, we show how to enforce the read-write file protocol on a distributed client as shown in Fig. 3. Suppose that the client stores three versions of the file (V_0, V_1, V_2) , and the file content is in the form of $[Cipher, Sig]$. To read file V_2 , DVAC obtains the ID^f and queries the metadata of the corresponding version from the smart contract. After obtaining the metadata, DVAC first verifies the signature Sig_2 according to the write access policy AP_w . If the verification is successful, $User_2$ has the write permission and the file version V_2 is valid.

Next, DVAC will decrypt $Cipher_2$ according to AP_r and the user's attribute key. If the user's attributes meet the access policy AP_r , the decryption is successful, and the plaintext file will be displayed in the client workspace. If the signature is invalid, $User_2$ does not have write permission. DVAC will return an error warning and choose to verify the latest historical version V_1 according to user requirements until a valid version is verified. If decryption fails, DVAC returns an error warning indicating no permissions. When a user modifies a file and intends to store it as a new version, DVAC encrypts the file according to the access policy and public parameter, and signs the ciphertext with the user's attribute key SK_w , obtaining $[Cipher_3, Sig_3]$. The user's local repository stores the V_3 version of the file, and can share it with other users.

The EAC protocol ensures that access control can be distributed on user nodes without the involvement of a central server, enforcing cryptographic protection. The data obtained from other nodes is in the form of $[Cipher, Sig]$. The plaintext data can be read only after being

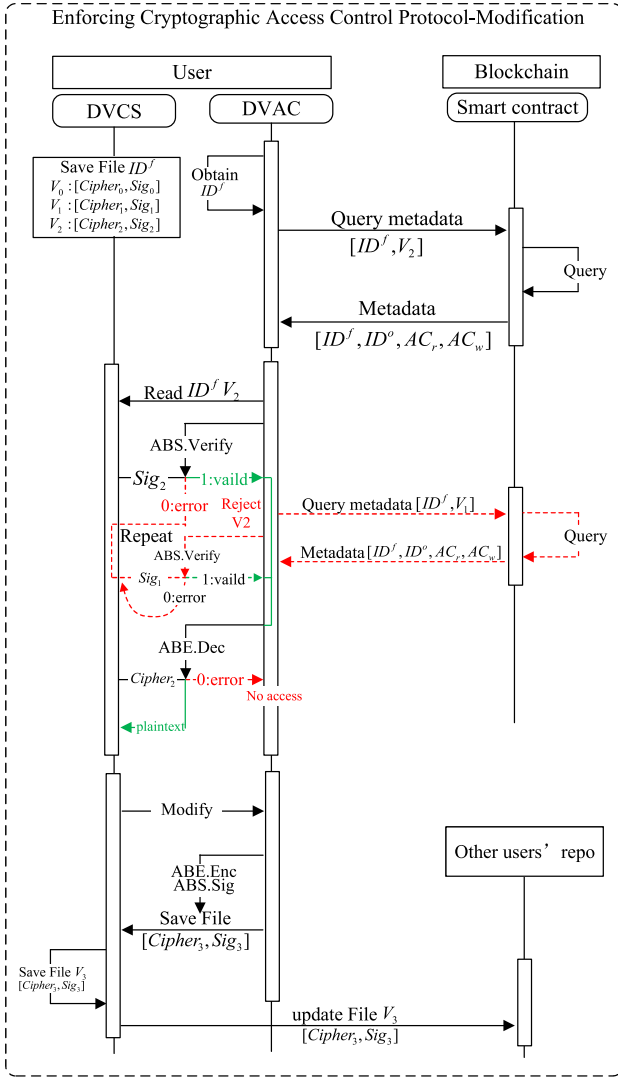


Fig. 3. Enforcing cryptographic access control protocol with other file operations.²

verified and decrypted by the user. Unauthorized modification can only affect the local repository of the modifier and cannot be accepted by other users.

4.3. Version control adaptation (VCA)

DVAC ensures that files in the user repository are encrypted. Users need to verify and decrypt $[Cipher, Sig]$ to obtain plaintext data. Each file version is encrypted with a distinct key, resulting in a different ciphertext. The version control function of DVCS, encompassing the maintenance, merging, and comparison of different versions, does not directly address the management of ciphertext. Consequently, we have developed an adaptation protocol for version control.

DVAC needs to release the cryptographic protection of related files when users execute version control commands and display the plaintext result of the command execution in the workspace. Meanwhile, the files in the repository remain encrypted. As shown in Fig. 4, the user's local repository stores three versions of file ID^f . The user initiates the diff/merge command for V_1 and V_2 in the workspace. Subsequently,

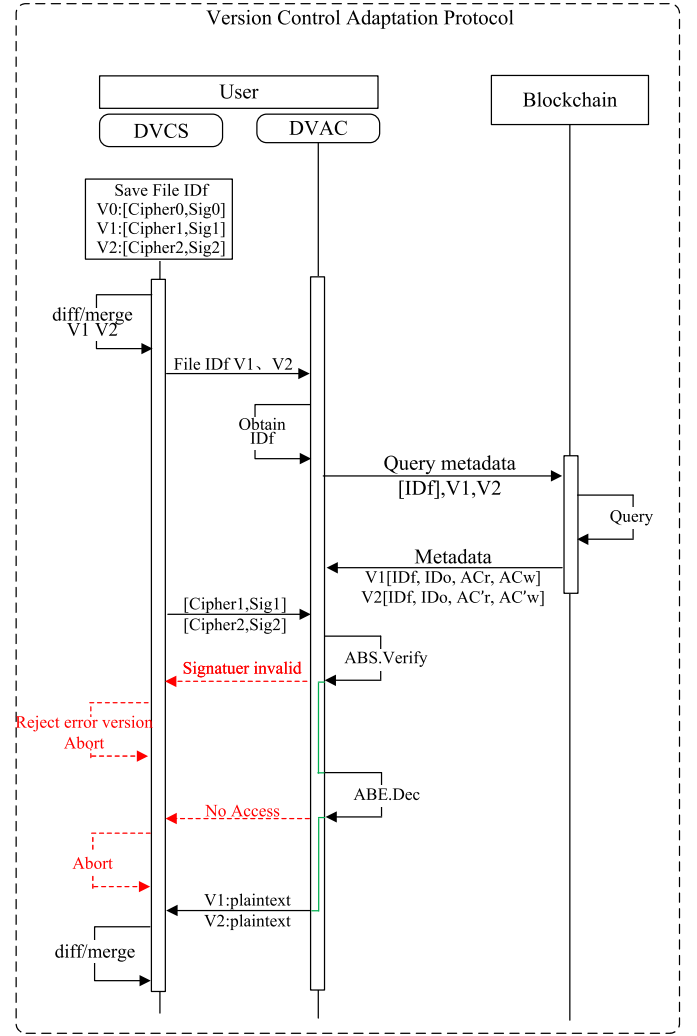


Fig. 4. Version control adaptation protocol.

DVAC sends a metadata query request to the smart contract with ID^f , V_1 , and V_2 as additional information. Upon receiving the metadata corresponding to the two versions from the smart contract, DVAC retrieves $[Cipher_1, Sig_1]$ and $[Cipher_2, Sig_2]$ from the user repository, verifying and decrypting the files individually. In the event of a verification or decryption failure, a warning message is sent to the user workspace. DVAC then forwards the successful plaintext of V_1 and V_2 to the DVCS for automatic diff/merge operation, and the final result is displayed in the workspace.

4.4. Trusted metadata management (TMM)

As described in Section 4.1, DVAC metadata contains information about files, as well as read and write access policies. The metadata is mapped to a file using a unique ID^f . Different versions of the file may correspond to different versions of the metadata. The security of metadata determines the effectiveness of access control. However, current DVCS lack a proper mechanism to store and maintain DVAC-related metadata for each file version. Once the maintenance of metadata has security vulnerabilities, it will pose additional risks to the security of DVCS data.

In a distributed scenario, if metadata is stored in DVCS in the same form as normal files, users cannot build a global view of the repository and are vulnerable to **metadata replay attacks**. The attacker may attempt to obtain revoked permissions, by replaying old

² V_0 is created by the data owner. Assume that V_1 is the version modified by $User_1$ and V_2 is the version modified by $User_2$.

metadata in which his permissions are not revoked. The victim, who has not received the latest version of the data, may accept this metadata and the attacker's revisions. Due to the inability of users to have a comprehensive overview of all file metadata, obtaining the required correct version of metadata independently is challenging. Additionally, metadata relies on ID^f for file mapping and ID^o for determining the data owner. Attackers may create a large number of files in advance and delete them to **preempt data owner attacks**. Even if the ID^f is uniquely generated, there is no way to avoid the extreme situation where two data owners create files with the same name and the same path in one repository, leading to the problem of confusing the ownership of the data. The fundamental cause of these attacks is the inability of the data owner and user to build a global view of the repository to check if ID^f has been allocated and obtain the corresponding valid metadata. However, building a global view will introduce the significant performance overhead, which becomes an obstacle in applying DVAC in the practical DVCS.

In order to solve the above problem, we have devised a blockchain-based mechanism for maintaining metadata, ensuring distributed and trustworthy management for DVAC-related metadata. The characteristics of blockchain, such as addition-only, immutability, and distributed ledger, are highly suitable for fully distributed environments. By leveraging smart contracts, we have developed a distributed and trusted mechanism for managing metadata.

As shown in Fig. 5, the TMM mechanism encompasses four algorithms: metadata creation, metadata modification, owner information replacement, and metadata querying.

Users are required to submit a request $Req(function, \dots)$ to the smart contract, with the first parameter specifying the action the smart contract should undertake (register, create, modify, replace or query).

The smart contract includes two fundamental functions: $Exist(a, List)$ to check if a exists in $List$, and $Get(a, List)$ to obtain an item based on the index of a in $List$.

The creation $Md_Create()$ and modification

$Md_Modify()$ of metadata necessitate the smart contract to validate the ID^o and the correctness of the account binding before appending the new metadata to $List_{AP}$, as shown in Fig. 5. The distinction lies in the fact that metadata creation requires the retrieval of $List_f$ to ensure that the ID^f is unused, thus avoiding metadata conflicts and resisting metadata preemption attacks. Owner information replacement $Md_Replace()$ is divided into ID^o replacement and bound account Acc^o replacement, denoted by a tag . Metadata querying $Md_Query()$ does not mandate authentication of the requester's identity, and the smart contract only indexes and outputs the metadata based on ID^f and version v .

5. Disac applied in git

DVAC can be readily applied in DVCS. In this context, we use the widely adopted DVCS, Git, as an example to illustrate the practical integration process. We introduce the integration of DVAC in Git and elucidate the specific process for users utilizing Git.

For Git integration, we have implemented a Git filter driver, and associated it with a long-running filter [36] for a specified subset of files by configuring Git attributes for the repository. The long-running filter, supported since Git 2.11, transmits the file path and content to the driver, and allows a long-running process to operate multiple files instead of only one in the previous version of Git filter. The registered filter driver includes the processes for two commands: clean and smudge.

- The clean command is invoked when a user's modification is added from the workspace to the stage through `git add`. In DVAC, as shown in Fig. 6, the process for clean command initially computes the ID^f , retrieves the metadata of ID^f by querying the smart contract, performs $ABE.Enc$ and $ABS.Sign$ with AP_r and AP_w , and ultimately returns the ciphertext and signature to the Git client for storage in the stage.

Metadata Management

$Meta_f \leftarrow Md_Create(ID^o, ID^f, AP_r^f, AP_w^f, Acc')$:

1. Query $ID^o, (Acc^o, ID^o) \leftarrow Get(ID^o, List_u)$;
2. If $Acc' = Acc^o$, continue, otherwise output "error 1";
3. Query $ID^f, 0/1 \leftarrow Exist(ID^f, List_f)$. If 0 is returned, output "error 2". If 1 is returned, $ID^f \rightarrow List_f$.
4. $(ID^f, ID^o, AP_r^f, AP_w^f) \rightarrow List_{AP}$, output $Meta_f = (ID^f, ID^o, AP_r^f, AP_w^f)$.

$Meta_f \leftarrow Md_Modify(ID^o, ID^f, AP_r^f, AP_w^f, v, Acc')$:

1. Query $ID^o, (Acc^o, ID^o) \leftarrow Get(ID^o, List_u)$;
2. If $Acc' = Acc^o$, continue, otherwise output "error 1";
3. Obtain $Meta_f$ of version v , $Meta_f \leftarrow Get(ID^f, List_{AP}, v)$;
4. $AP_r^f, AP_w^f \rightarrow Meta_f$, output new $Meta_f$.

$Meta_f \leftarrow Md_Replace(ID^f, ID^o, ID^{o'} | Acc^{o'}, tag)$:

1. Query $ID^o, (Acc^o, ID^o) \leftarrow Get(ID^o, List_u)$
2. If $Acc' = Acc^o$, continue, otherwise output "error1";
3. $tag = 1: (Acc^{o'}, ID^o) \rightarrow List_u$
4. $tag = 2$:
 - 1) Obtain $Meta_{ID^f}$ of version v , $Meta_f \leftarrow Get(ID^f, List_{AP}, v)$;
 - 2) $ID^{o'} \rightarrow Meta_f$.

$Meta_f \leftarrow Md_Query(ID^f, v)$:

1. Obtain $Meta_f$ of version v , $Meta_{ID^f} \leftarrow Get(ID^f, List_{AP}, v)$;
2. Output $Meta_f$.

Fig. 5. Metadata management algorithms.

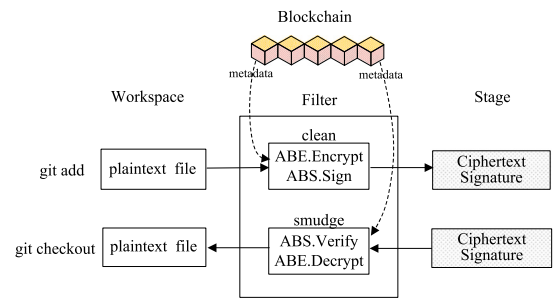


Fig. 6. DVAC design integrated with Git.

- The smudge command is invoked when a user pulls the revisions from the remote/local repository or stage to the workspace through the commands such as `git pull` and `git checkout`. In DVAC, as shown in Fig. 6, the process for the smudge command derives ID^f , queries the smart contract for the latest metadata, verifies the ABS signature with AP_w , obtains the plaintext after performing ABE decryption with AP_r , and finally returns the plaintext file to the workspace.

We have developed a web interface based on web3.js [37] for the data owner to manage the metadata. This includes the Md_Create function for establishing the initial access control policy for a new ID^f , the Md_Modify function for altering the access control policy, the

Md_Replace function for transferring ownership and the account, and the *Md_Query* function for retrieving the correct metadata. For users, the data exists in plaintext in the workspace, and the VCA protocol ensures that standard Git operations (such as `git merge` or conflict resolution) remain unaffected. For the `git rm` operation, instead of deleting the file, DVAC clears the file content and stores the ABS signature (including the parameters) of the `git rm` operation in the file, which allows other users to calculate ID^f and check the write permission. The encrypted and signed contents of the file are ultimately stored in the *.objects* folder of the repository.

6. Security analysis

In this section, we present a concise security analysis of DVAC.

Theorem 1. *DVAC provides file granularity, read permission security for unauthorized read users and write permission security for unauthorized write users.*

Proof. In a file f [$Meta_f$, $Cipher_f$, Sig_f], the attribute set S_a of unauthorized read user A does not meet AP_r , and the attribute set S_b of unauthorized write user B does not meet AP_w . After obtaining $Cipher_f$, user A needs to decrypt the $Cipher_f$ to obtain the plaintext. Because S_a does not satisfy AP_r and returns an error when executing *ABE.Dec*, plaintext cannot be obtained by user A . User B reads file f and makes modifications to obtain file f' . When *ABS.Sign* is executed based on AP_w in $Meta_f$, an error is returned because S_b does not meet AP_w , and the correct signature cannot be obtained. However, file f' [$Cipher_{f'}$, $Sig_{f'}$] is generated if user B chooses a forged (S_b satisfied) AP'_w to sign, or a forged signature. Other users obtain the correct AP_w through the smart contract after obtaining the file f' , and return 0 after executing *ABS.Verify*($Sig_{f'}$, AP_w). Therefore, all users who execute DVAC correctly will not accept the file f' , meaning that the modifications made by unauthorized write user B are invalid. $\square$

Theorem 2. *DVAC is resistant to data owner preemption and impersonation attacks targeting metadata.*

Proof. Against the two types of attacks:

- For the data owner preemption attack, the attacker will create a large number of file metadata in advance to overwrite the ID^f that may be used to become the corresponding data owner in advance. In DVAC, the creation of metadata must make a *Req(create)* to the smart contract. In the process of calling *Md_Create*, it is necessary to determine whether the created ID^f has been recorded in the $List_f$. Thus, the uniqueness check in *Md_Create* ensures that the invoker will select a globally unique and unused ID^f , making preemption of ID^f impossible. In addition, metadata creation requires a transaction to be initiated and costs a certain amount of ETH, introducing the significant overhead for the attackers attempting frequent invocations to preempt a larger number of ID^f .
- For a data owner impersonation attack, an attacker may modify the ID^o in the metadata with *Md_Replace*, or by directly tampering with the blockchain. For *Md_Replace*, as long as the user's private key remains secure, the attacker cannot forge a valid signature to impersonate the victim, and therefore the consistency check of the invoker's identifier and stored ID^o in *Md_Replace* prevents the attacker from successfully invoking *Md_Replace*. Direct tampering on the blockchain is prevented by the tamper-resistant feature of the smart contract (with the underlying blockchain).

Therefore, DVAC can resist data owner preemption and impersonation attacks targeting metadata. $\square$

Theorem 3. *DVAC can protect metadata from tampering and resist metadata replay attacks.*

Proof. The attackers may attempt to tamper with AP_r or AP_w in the metadata to gain more privileges over the victim's data. To tamper with the metadata, the attacker may choose to modify it by invoking *Md_Modify*, directly tamper with AP_r and AP_w stored in the blockchain, or replay the AP_r and AP_w .

- For *Md_Modify*, DVAC requires the invoker's identifier to match the stored ID^o for ID^f , and the attacker cannot forge a valid signature corresponding to the public key for the stored ID^o . Consequently, the attacker cannot successfully impersonate the victim to invoke *Md_Modify*.
- Direct modifications to AP_r and AP_w are also infeasible due to the tamper-resistant nature of the blockchain.
- Metadata replay attacks cannot succeed either. The attackers cannot perform the read (write) operation once the corresponding permission is revoked, as correct users can obtain a global view of the metadata for ID^f easily through the query interface of the smart contract, so they are therefore able to obtain the correct metadata for each version of the data.

Therefore, DVAC can protect metadata from tampering and resist metadata replay attacks. $\square$

In addition, the anti-collusion ability of DVAC depends on the anti-collusion ability of ABE and ABS algorithms. The data owner can easily manage the metadata, while the user can obtain any valid metadata through the query interface, without any extra user-side verification.

7. Evaluation

In this section, we provide the implementation details and performance evaluation of DVAC.

7.1. Functional comparison

We conducted a functional comparison of DVAC with existing access control solutions for Git, evaluating them based on five aspects: control granularity of read and write permissions, key maintenance overhead, distributed scenario support, and version control adaptation. Our findings indicate that the current Git access control schemes lack the ability to offer fine-grained write access control and struggle to support distributed scenarios, as shown in Table 2.

Both `git-remote-gcrypt` [18] and `Git-secret` [38] solely offer encryption protection at the repository level and do not support write access control. They utilize `GnuPG` (GPG) [39] to maintain the public key of all users and provide distribution of symmetric keys. While `git-remote-gcrypt` uses AES-128 for data encryption, `git-crypt` [17], similar in nature, employs AES-256 (in CTR mode) for data encryption and can provide file-granular encryption protection. None of these solutions support write access control, and the public key maintained by GPG is either stored on the public key server or maintained locally by the user. The $O(N_u)$ key maintenance overhead also limits the expansion of distributed scenarios.

`GitHub Enterprise` [34] provides access control services based on the role-based access control (RBAC) model, offering repository-level read access control and branch-level write access control. However, the implementation of access control requires verification by the `GitHub` server. `Keybase-git` [35] is built on the `Keybase Encrypted File System` (KBFS), and uses the ACL model for access control. Users are also required to maintain N_u public key information. While these approaches encrypt data during file transfer, the files remain in plaintext in the user's local repository, thus obviating the need for version control adaptation. Nevertheless, this method cannot guarantee the security of the user's local data, leaving it vulnerable to potential leakage.

Table 2
Comparison of access control solutions for Git.

	git-remote-gcrypt	Git-secret	git-crypt	GitHub Enterprise	Keybase-git	Gringotts	DVAC in Git
read permission control ^a	R	R	F	R	R	F	F
write permission control ^a	✗	✗	✗	B	R	B	F
key maintenance ^b	$O(N_u)$	$O(N_u)$	$O(N_u)$	$O(N_u)$	$O(N_u)$	$O(N_u)$	2
Distributed scenario ^c	✗	✗	✗	✗	✗	✗	✓
Version control adaptation	–	–	–	–	–	✓	✓
Data in repos ^d	P	P	P	P	P	C	C+S

^a Read/write permission control. “✗”: not supported; “R”: repository level; “B”: branch level; “F”: file level.

^b Key maintenance. “ N_u ”: the number of users.

^c Distributed scenario: no centralized server for file storage or access control implementation. “✗”: not supported or excessive encryption overhead; “✓”: support and acceptance of expenses.

^d Data in repos. “P”: plaintext; “C”: ciphertext; “C+S”: ciphertext+signature.

Gringotts [19] adopts ABE to encrypt files in the repository, and uses the elliptic curve digital signature algorithm (ECDSA) to provide branch-level write permission control. While ABE allows users whose attributes meet the access policy to decrypt files, the remote server still needs to verify the signature to enforce write access control and maintain N_u user public keys. In the distributed scenario, there is no central server for access control implementation and file storage, and it is difficult to directly apply to existing DVCS systems (such as Git). In addition, coarse-grained write access control at the branch level requires users to manually generate branches and merge them, which seriously affects user experience in fully distributed scenarios. In fact, Gringotts is a targeted version control system with access control deployed, and its version control function is closely related to the access control function, so the scheme cannot be easily migrated to other DVCS, such as Git. So the above solutions make it difficult to complete the decentralized deployment and application in the DVCS.

DVAC, integrated with Git, adopts ABE and ABS algorithms to provide file-granularity read and write access control, enabling fine-grained data protection. Through attribute-based cryptographic algorithms, the encryption and signature are calculated according to the attributed access policy. Unlike traditional public key cryptographic algorithms, there is no need to perform encryption for each user and maintain a large number of user public keys. Each user only maintains their own ABE and ABS private keys. Furthermore, DVAC’s read and write access control can be fully implemented on the client side, transparent to users, without the need for a central server. It supports distributed scenarios and is well-suited for distributed version control systems such as Git.

7.2. Performance

We have evaluated the performance overhead of enforcing read and write access control introduced by DVAC and compared it to existing schemes.

Setting. We have implemented DVAC and integrated it with the Git client. DVAC relies on ABE and ABS to enforce the access control, and integrates with Git through Git filters, i.e., using a clean module for ABE encryption and ABS signing before sharing data, and a smudge module to verify (ABS) signature and (ABE) decrypt the obtained data. The Git client integrated with DVAC was deployed in a MacBook Pro laptop with 4×2.3 GHz Intel Core i7-1068NG7 CPUs and 16 GB 3733 MHz LPDDR4X RAM. The metadata were stored in the Ethereum testnet Ropsten [40]. We adopt the ABE algorithm [41] and ABS algorithm [25] in the prototype implementation, and implement them using C language based on the PBC (pairing-based cryptography) library [42].

To evaluate the performance overhead in the practical environment, we measured the introduced latency and storage for different access control policies (one with 5 attributes and the other with 10) and file sizes. The range of file size was set as 0–1 MB, according to our statistics on GitHub dataset with Google BigQuery [43], which is based on the GitHub Archive Project [44]. We analyzed the complete snapshots of the content of more than 2.8 million open-source GitHub repositories until 19 August 2023. As shown in Table 4, 77.44% files are smaller than 10KB, and 99.42% files are smaller than 1MB.

7.2.1. Comparison

We compared the performance of DVAC with existing Git access control schemes Gringotts [19] and git-crypt [17].

We selected the top 10 repositories with the most stars on GitHub for the experiment. To measure the commit delta time, we replayed 5000 commits in the new repository. DVAC encrypts and signs during git add, so the extra time includes measuring the time to git add all changes in a commit. For Gringotts and git-crypt, this includes the extra time spent encrypting and decrypting files in a commit. Regarding checkout time, we git checkout the first 5000 commits in the clone’s repository and measure the extra time. All repositories were configured with five access policies, each containing 10 attributes in DVAC and Gringotts, and a coarse-grained access policy for 10 users in git-crypt.

From Table 3, we can see that the time overhead used by DVAC is close to but slightly lower than git-crypt, and the overhead is higher than Gringotts. This is because Gringotts uses ECDSA for branch-level write access control, while DVAC employs ABS for finer file-level write access control, offering a better user experience for DVCS. Specifically, git add requires ABS signature, and git checkout requires an ABS verification, thus bringing some overhead, which is also explained in detail below. As shown in Section 7.1, unlike Gringotts, DVAC operates without the need for a central server, enabling fully distributed access control. Furthermore, the file-level read and write access control limits permissions to each file version, ensuring that user collaboration, including merge operations, is not disrupted. This also facilitates the implementation of subsequent permission revocations and other policies, allowing DVAC to be used as a plug-in for various DVCS. According to the functional comparison in Table 2, DVAC maintains a time cost within the millisecond range, providing the aforementioned security features without affecting the overall user experience. Overall, DVAC minimizes changes to existing DVCS (such as Git) while implementing distributed mandatory read and write access control independent of a central server, and provides a more adaptable and secure access control scheme for DVCS at an acceptable cost.

7.2.2. Latency

We measured the latency for git add passing through the clean module and git checkout passing through the smudge module for different file sizes and access control policies. As shown in Fig. 7, for an access control policy with five attributes, the processing time is less than 78 ms and 43 ms for git add and checkout with a 1 MB file, respectively, and the processing time reduced to 67 ms and 33 ms for a 10 KB file. For an access control policy with 10 attributes, the processing time increases slightly, to 94 ms and 46 ms for git add and checkout with a 10 KB file, respectively.

In the measurement, the default access policy is not modified. It is assumed that the access policy is cached locally after the metadata is obtained from the blockchain for the first time.

The introduced latency consists of three parts: ABE and ABS processing, metadata querying and file transmission between the Git client and the Git filter driver.

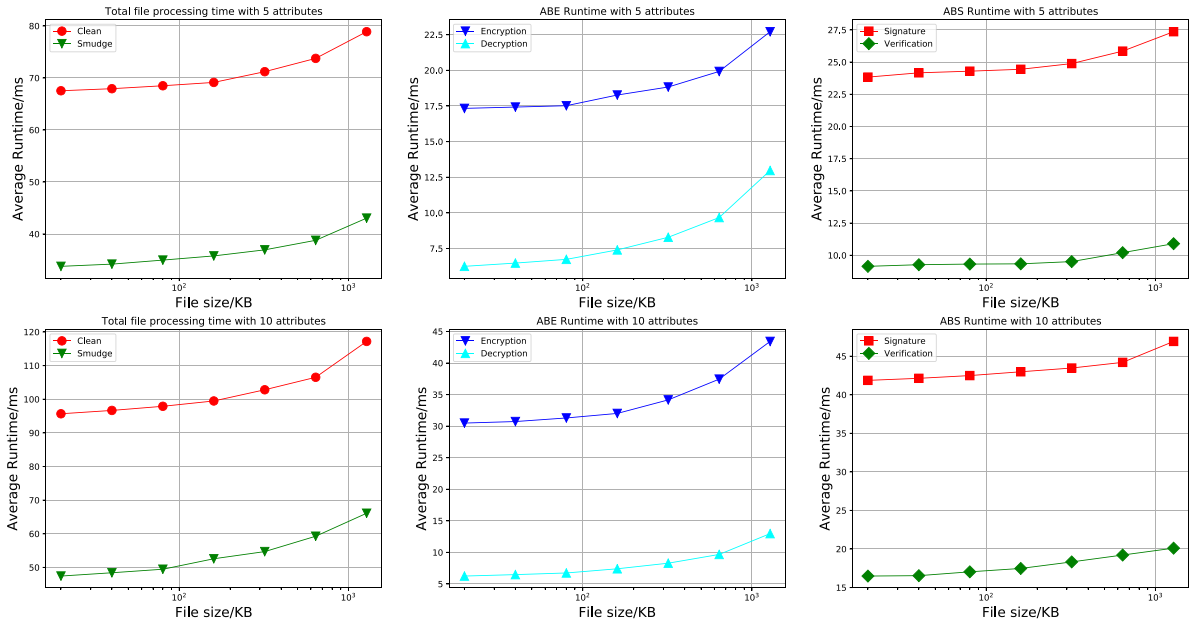


Fig. 7. The total file processing time.

Table 3
Time delta overhead for committing and checking files.

	Amortized commit time difference(s)			Amortized checkout time difference(s)		
	DVAC(git)	gringotts	git-crypt	DVAC(git)	gringotts	git-crypt
bootstrap	0.256	0.16	0.125	0.128	0.042	0.106
d3	0.238	0.168	0.188	0.155	0.056	0.151
electron	0.217	0.167	0.134	0.143	0.056	0.094
flutter	0.249	0.161	0.179	0.152	0.091	0.145
linux	0.252	0.17	0.337	0.198	0.207	0.224
ohmyzsh	0.221	0.167	0.097	0.113	0.042	0.017
react	0.234	0.184	0.13	0.125	0.068	0.059
tensorflow	0.371	0.243	1.061	0.271	0.216	0.421
vscode	0.218	0.14	0.174	0.184	0.135	0.141
vue	0.249	0.159	0.125	0.112	0.036	0.091
Average	0.2505	0.1719	0.255	0.1581	0.0949	0.1449

Table 4
File size statistics on Github (until 2023.8.19).

File size	Number	Percentage
0-10KB	205563927	77.44%
10KB-1MB	57 544 225	21.98%
1M-10M	2097 378	0.79%
10M+	236 104	0.09%

ABE & ABS processing. ABE and ABS processing requires more than 50% of the whole processing time. As shown in Fig. 7, we measure the time overhead of ABE and ABS in DVAC for files with different sizes and access policies with 5 or 10 attributes. The ABE part contains AES-128 symmetric encryption. ABE and ABS processing time increases with file size, and the ABE processing time increases more obviously because the file content needs to be encrypted. ABS only hashes the contents of the file and is relatively unaffected.

ABE encryption and ABS processing time also increase with the number of attributes of the policy and user, and ABE decryption time is not affected by attributes and policies. For example, for a used 10 KB file, the time for ABE encryption and ABS signature increases from 40 ms to 70 ms when the number of attributes in the policy increases from 5 to 10. The `git add` operation needs more time than `git pull`, as ABE encryption and ABS signing require more exponentiations than ABE decryption and ABS verification.

Metadata querying. DVAC synchronizes blocks of the Ethereum testnet and queries the metadata locally, which requires about 80 ms when the number of blocks is about 500,000. However, the synchronization of blocks introduces a significant storage overhead; for example, 75 GB is required for 500,000 blocks. To eliminate the storage requirement on the client, DVAC supports to query the metadata through a third-party service (e.g., Infura [45]), which needs 100 ms in our network environment. In addition, the efficiency of metadata querying can also be further improved through methods such as [46].

The transfer latency of metadata creation and modification is similar to the latency of metadata acquisition, both of which are relatively infrequently used and performed by the data owner through web interfaces without integration into the Git client.

File transmission between Git client and git filter driver. In DVAC, the transmission of files between the Git client and the Git filter driver occurs using the long-running process protocol. The latency for file transmission is contingent upon the file size, ranging from less than 7 ms for a 1 MB file to 1500 ms for a 10 MB file, and escalating to 98,000 ms for a 100 MB file (the maximum file size in GitHub). Nevertheless, given that sensitive data is typically stored in a separate file and 99.42% of files are less than 1 MB, the latency for file transmission remains within acceptable bounds.

7.2.3. Storage

DVAC introduces two extra storage requirements for the Git client. Firstly, each client needs to store the blocks synchronized from the

Ethereum testnet, which amounts to 75 GB when the number of blocks reaches approximately 500,000. This storage overhead can be eliminated by using a third-party service (e.g., Infura [45]) for metadata querying. Secondly, each file is stored in the form of ABE ciphertext and ABS signature, rather than the original data. In our setting, this necessitates approximately 5.16 KB of extra storage for each file when a 10-attribute access control policy is employed.

7.2.4. ETH overhead

In the metadata management part, DVAC need to interact with smart contracts, which inevitably involves the overhead of publishing transactions. In the experimental environment, an upload transaction of metadata requires about ETH 0.0005, while a query of metadata does not cost Ether. In addition, if DVCS is built on the basis of multiple alliances or organizations, metadata management can be carried out on the alliance chain, which can greatly reduce the transaction overhead while improving security. Therefore, the overhead of Ether is very small.

8. Discussion

DVAC controls read and write permissions using ABE and ABS, respectively, while adapting to DVCS characteristics. It is not tied to any specific DVCS but addresses universal version control and metadata management needs. We developed version control and metadata protocols tailored to Git, enhancing DVAC's compatibility with it. However, this approach is also applicable to other DVCS systems, which all require version control and distributed security management. Adapting DVAC to different DVCS involves adjusting based on each system's characteristics, primarily affecting the configuration environment and metadata management deployment, as discussed in Section 5.

Regarding attribute-based signcryption algorithms (ABSC), most existing ABSC algorithms support only a single policy mode, such as ciphertext-policy or key-policy. Hybrid schemes [47,48] combine KP-ABS with CP-ABE but still use a single access policy during the signcryption phase, and generate signature keys based on signature access policies, leading to the need for users to maintain multiple signature keys, especially with varying write permissions across files. In DVAC, we bind signature and decryption keys to attributes separately and bind signatures and ciphertexts to read and write access policies per file. This minimizes user key maintenance overhead and facilitates file permission control. Thus, although ABSC algorithms offer higher computational efficiency and lower communication costs, due to sacrificing some flexibility, existing ABSC algorithms cannot be directly applied to DVAC. Nevertheless, ABSC in principle supports the extension of different access policies, and its integration concept provides us with future optimization directions, so we will consider the possibility of combining ABE with ABS further improve computing efficiency in the future.

9. Conclusion

In this paper, we propose DVAC, an enforcing cryptographic access control scheme for fully distributed VCS, aimed at preventing unauthorized read or write access to sensitive data in distributed repositories. According to the operation requirements and version control characteristics of DVCS, DVAC sets the enforcing cryptographic access control protocol and version control adaptation protocol. DVAC implements access control on distributed nodes, protecting data access without relying on a central server, and DVAC's architecture integrates seamlessly with existing DVCS, necessitating only a few straightforward configurations to ensure compatibility. The participation of smart contracts guarantees that only the data owner retains complete control over the shared data and the consistency and security of multi-node access control. We have implemented DVAC, integrated it with Git, and the performance

evaluation demonstrates that the introduced overhead is acceptable in practical environment.

Our current scheme also has some limitations, such as the high computational cost of attribute-based cryptography and the undiscussed fine-grained permission dynamic management. Future research will focus on optimizing distributed system access control, such as multi-authority fine-grained retractions, and developing smart contract-assisted computing schemes for resource-limited devices, in order to improve system efficiency and the scope of application.

CRedit authorship contribution statement

Xin Xu: Writing – original draft, Software, Methodology. **Zhen Yang:** Writing – review & editing, Validation. **Quanwei Cai:** Writing – review & editing, Validation. **Jingqiang Lin:** Resources, Methodology. **Liangqin Ren:** Software. **Bo Chen:** Investigation. **Yongfeng Huang:** Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Yongfeng Huang reports financial support was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by National Natural Science Foundation of China (No. 82090053). The authors wish to thank anonymous reviewers for their valuable comments and suggestions that improved this paper.

Data availability

Data will be made available on request.

References

- [1] CVS. Concurrent versions system. 2019, Retrieved from <https://www.nongnu.org/cvs/>. [Accessed 29 February 2024].
- [2] SVN. Apache subversion. 2023, Retrieved from <https://subversion.apache.org/>. [Accessed 29 February 2024].
- [3] Git. distributed-even-if-your-workflow-isnt. 2024, Retrieved from <https://git-scm.com>. [Accessed 29 February 2024].
- [4] Veracity. An open source, distributed version control and bug tracking system for Windows, Mac OS X, and Linux. 2014, Retrieved from <http://veracity-scm.com/>. [Accessed 29 February 2024].
- [5] Ruj S, Stojmenovic M, Nayak A. Privacy preserving access control with authentication for securing data in clouds. In: International symposium on cluster, cloud and grid computing. 2012, p. 556–63.
- [6] Thwin TT, Vasupongayya S, et al. Blockchain-based access control model to preserve privacy for personal health record systems. Secur Commun Netw 2019;2019.
- [7] Wang S, Wang X, Zhang Y. A secure cloud storage framework with access control based on blockchain. IEEE Access 2019;7:112713–25.
- [8] Li X, Wang H, Ma S, Xiao M, Huang Q. Revocable and verifiable weighted attribute-based encryption with collaborative access for electronic health record in cloud. Cybersecurity 2024;7(1):1–19.
- [9] Patil RY. A secure privacy preserving and access control scheme for medical internet of things (MIoT) using attribute-based signcryption. Int J Inf Technol 2024;16(1):181–91.
- [10] Shafagh H, Burkhhalter L, Ratnasamy S, Hithnawi A. Droplet: Decentralized authorization and access control for encrypted data streams. In: 29th {USENIX} Security Symposium. 2020, p. 2469–86.
- [11] Yang Z, Chen X, He Y, Liu L, Che Y, Wang X, et al. An attribute-based access control scheme using blockchain technology for IoT data protection. High- Confid Comput 2024;4(3):100199.

- [12] Li X, Wang H, Ma S. An efficient ciphertext-policy weighted attribute-based encryption with collaborative access for cloud storage. *Comput Stand Interfaces* 2025;91:103872.
- [13] Zhang Y, He D, Choo K-KR. BaDS: Blockchain-based architecture for data sharing with ABS and CP-ABE in IoT. *Wirel Commun Mob Comput* 2018;2018.
- [14] Exceline CE, Nagarajan S. Flexible access control mechanism for cloud stored EHR using consortium blockchain. *Int J Syst Assur Eng Manag* 2024;15(1):503–18.
- [15] Xu X, Cai Q, Lin J, Pan S, Ren L. Enforcing access control in distributed version control systems. In: 2019 IEEE international conference on multimedia and expo. IEEE; 2019, p. 772–7.
- [16] sitaramc. Gitolite. 2014, Retrieved from <http://gitolite.com/>. [Accessed 29 February 2024].
- [17] Ayer A. Git-crypt. 2012, Retrieved from <https://www.agwa.name/projects/git-crypt/>. [Accessed 29 February 2024].
- [18] Bluss. Git-remote-gcrypt. 2021, Retrieved from <https://spwhitton.name/tech/code/git-remote-gcrypt/>. [Accessed 29 February 2024].
- [19] Xu W, Ma H, Song Z, Li J, Zhang R. Gringotts: An encrypted version control system with less trust on servers. *IEEE Trans Dependable Secur Comput* 2023.
- [20] Sahai A, Waters B. Fuzzy identity-based encryption. In: Annual international conference on the theory and applications of cryptographic techniques. Springer; 2005, p. 457–73.
- [21] Goyal V, Pandey O, Sahai A, Waters B. Attribute-based encryption for fine-grained access control of encrypted data. In: ACM conference on computer and communications security. 2006, p. 89–98.
- [22] Bethencourt J, Sahai A, Waters B. Ciphertext-policy attribute-based encryption. In: IEEE symposium on security and privacy. IEEE; 2007, p. 321–34.
- [23] Yin S, Li H, Teng L, Laghari AA, Estrela VV. Attribute-based multiparty searchable encryption model for privacy protection of text data. *Multimedia Tools Appl* 2024;83(15):45881–902.
- [24] Huang M, Liu Y, Yang B, Zhao Y, Zhang M. Efficient revocable attribute-based encryption with data integrity and key escrow-free. *Information* 2024;15(1):32.
- [25] Maji HK, Prabhakaran M, Rosulek M. Attribute-based signatures. In: International conference on topics in cryptology: CT-RSA. 2011.
- [26] Smart Contracts. Smart contracts. 2022, Retrieved from <http://szabo.best.vwh.net/smart.contracts.html>. (Accessed 29 February 2024).
- [27] Christidis K, Devetsikiotis M. Blockchains and smart contracts for the internet of things. *IEEE Access* 2016;4:2292–303.
- [28] Wood G, et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Proj Yellow Pap* 2014;151(2014):1–32.
- [29] hyperledger fabric. A blockchain platform for the enterprise. 2023, Retrieved from <https://hyperledger-fabric.readthedocs.io/>. [Accessed 29 February 2024].
- [30] Vimercati SDCD, Foresti S, Jajodia S, Paraboschi S, Samarati P. Over-encryption: Management of access control evolution on outsourced data. In: International conference on very large data bases. 2007, p. 123–34.
- [31] Di Vimercati SDC, Foresti S, Jajodia S, Livraga G, Paraboschi S, Samarati P. Enforcing dynamic write privileges in data outsourcing. *Comput Secur* 2013;39(4):47–63.
- [32] Zhao F, Nishide T, Sakurai K. Realizing fine-grained and flexible access control to outsourced data with attribute-based cryptosystems. In: International conference on information security practice and experience. 2011, p. 83–97.
- [33] slobobaby. Git filter. 2014, Retrieved from <https://github.com/slobobaby/git-filter>. [Accessed 29 February 2024].
- [34] GitHub Enterprise. A smarter way to work together. 2022, Retrieved from <https://github.com/enterprise>. [Accessed 29 February 2024].
- [35] Keybase. Keybase launches encrypted git. 2017, Retrieved from <https://keybase.io/blog/encrypted-git-for-everyone>. [Accessed 29 February 2024].
- [36] Git. Git attributes: long-running-filter. 2024, Retrieved from <https://git-scm.com/docs/gitattributes>. [Accessed 29 February 2024].
- [37] ChainSafe Systems. Web3.js. 2024, Retrieved from <https://github.com/ethereum/web3.js/>. [Accessed 29 February 2024].
- [38] git-secret. Bash-tool to store your private data inside a git repository. 2022, Retrieved from <https://sobleevn.me/git-secret/>. [Accessed 29 February 2024].
- [39] GnuPG(GPG). The GNU privacy guard. 2024, Retrieved from <https://www.gnupg.org/>. [Accessed 29 February 2024].
- [40] Ropsten. TESTNET Ropsten (ETH) Blockchain Explorer. 2023, Retrieved from <https://ropsten.etherscan.io/>. [Accessed 29 February 2024].
- [41] Agrawal S, Chase M. FAME: fast attribute-based message encryption. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security. ACM; 2017, p. 665–82.
- [42] Lynn B. The PBC(Pairing-Based Cryptography) Library. 2006, Retrieved from <http://crypto.stanford.edu/pbc/>. [Accessed 29 February 2024].
- [43] Google Cloud. GitHub Activity Data. 2023, Retrieved from <https://console.cloud.google.com/marketplace/product/github/github-repos>. [Accessed 29 February 2024].
- [44] GHarchive. Github Archive. 2023, Retrieved from <https://www.gharchive.org/>. [Accessed 29 February 2024].
- [45] Infura. Unleash the full potential of Web3. 2024, Retrieved from <https://infura.io/>. [Accessed 29 February 2024].
- [46] Wu H, Peng Z, Guo S, Yang Y, Xiao B. VQL: Efficient and verifiable cloud query services for blockchain systems. *IEEE Trans Parallel Distrib Syst* 2022;33(6):1393–406.
- [47] Yu G, Cao Z. Attribute-based signcryption with hybrid access policy. *Peer-to-Peer Netw Appl* 2017;10:253–61.
- [48] Yu J, Liu S, Wang S, Xiao Y, Yan B. LH-ABSC: A lightweight hybrid attribute-based signcryption scheme for cloud-fog-assisted IoT. *IEEE Internet Things J* 2020;7(9):7949–66.